

Advice for applying machine learning

Deciding what to try next

Debugging a learning algorithm

You've implemented regularized linear regression on housing prices

$$\rightarrow J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

But it makes unacceptably large errors in predictions. What do you try next?

- $\rightarrow$ Get more training examples
- $\rightarrow$ Try smaller sets of features
- $\rightarrow$ Try getting additional features
- $\rightarrow$ Try adding polynomial features ($x_1^2, x_2^2, x_1x_2, etc$)
- $\rightarrow$ Try decreasing λ
- $\rightarrow$ Try increasing λ



Machine learning diagnostic

Machine learning diagnostic

Diagnostic:

A test that you run to gain insight into what is/isn't working with a learning algorithm, to gain guidance into improving its performance.

Diagnostics can take time to implement but doing so can be a very good use of your time.

Evaluating a model

Divide the dataset into 2 set: Training set and test set

Evaluating your model

Dataset:

	size	price	
70%	2104	400	training set → m_{train} = no. training examples $= 7$
	1600	330	
	2400	369	
	1416	232	
	3000	540	
	1985	300	
	1534	315	
30%	1427	199	test set → m_{test} = no. test examples $= 3$
	1380	212	
	1494	243	

$(x^{(1)}, y^{(1)})$
 $(x^{(2)}, y^{(2)})$
 $\vdots$
 $(x^{(m_{train})}, y^{(m_{train})})$

$(x_{test}^{(1)}, y_{test}^{(1)})$
 $\vdots$
 $(x_{test}^{(m_{test})}, y_{test}^{(m_{test})})$

Train/test procedure for linear regression (with squared error cost)

Fit parameters by minimizing cost function $J(\vec{w}, b)$

$$\rightarrow J(\vec{w}, b) = \left[\frac{1}{2m_{train}} \sum_{i=1}^{m_{train}} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m_{train}} \sum_{j=1}^n w_j^2 \right]$$

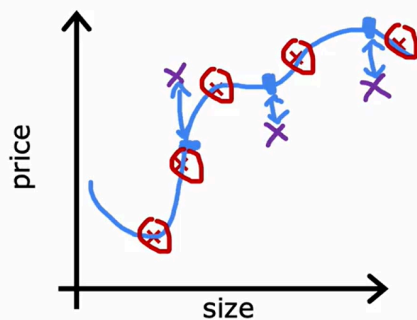
Compute test error:

$$J_{test}(\vec{w}, b) = \frac{1}{2m_{test}} \left[\sum_{i=1}^{m_{test}} (f_{\vec{w}, b}(\vec{x}_{test}^{(i)}) - y_{test}^{(i)})^2 \right] \quad \cancel{\sum_{j=1}^n w_j^2}$$

Compute training error:

$$J_{train}(\vec{w}, b) = \frac{1}{2m_{train}} \left[\sum_{i=1}^{m_{train}} (f_{\vec{w}, b}(\vec{x}_{train}^{(i)}) - y_{train}^{(i)})^2 \right]$$

Train/test procedure for linear regression (with squared error cost)



$x = \text{train}$

$x = \text{test}$

$J_{\text{train}}(\vec{w}, b)$ will be low

$J_{\text{test}}(\vec{w}, b)$ will be high

in the model like what we saw earlier in this video, $J(w, b)$ will be low because the average error on your training examples will be zero or very close to zero. So J_{train} will be very close to zero. But if you have a few additional examples in your test set that the algorithm had not trained on, then those test examples, might look like these. And there's a large gap between what the algorithm is predicting as the estimated housing price, and the actual value of those housing prices. And so, J_{test} will be high. So seeing that J_{test} is high on this model, gives you a way to realize that even though it does great on the training set, is actually not so good at generalizing to new examples to new data points that were not in the training set.

Train/test procedure for classification problem

0/1

Fit parameters by minimizing $J(\vec{w}, b)$ to find $\vec{w}, b$

E.g.,

$$J(\vec{w}, b) = -\frac{1}{m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} \left[y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m_{\text{train}}} \sum_{j=1}^n w_j^2$$

Compute test error:

$$J_{\text{test}}(\vec{w}, b) = -\frac{1}{m_{\text{test}}} \sum_{i=1}^{m_{\text{test}}} \left[y_{\text{test}}^{(i)} \log(f_{\vec{w}, b}(\vec{x}_{\text{test}}^{(i)})) + (1 - y_{\text{test}}^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}_{\text{test}}^{(i)})) \right]$$

Compute train error:

$$J_{\text{train}}(\vec{w}, b) = -\frac{1}{m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} \left[y_{\text{train}}^{(i)} \log(f_{\vec{w}, b}(\vec{x}_{\text{train}}^{(i)})) + (1 - y_{\text{train}}^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}_{\text{train}}^{(i)})) \right]$$

Now, let's take a look at how you apply this procedure to a classification problem.

For example, if you are classifying between handwritten digits that are either 0 or 1, so same as before, you fit the parameters by minimizing the cost function to find the parameters w, b . For example, if you were training logistic regression, then this would be the cost function J of w, b , where this is the usual logistic loss function, and then plus also the regularization term. And to compute the test error, J_{test} is then the average over your test examples, that's that 30% of your data that wasn't in the training set of the logistic loss on your test set. And the training error you can also compute using this formula, is the average logistic loss on your training data that the algorithm was using to minimize the cost function J of w, b . Well, when I described here will work, okay, for figuring out if your learning algorithm is doing well, by seeing how I was doing in terms of test error. When applying machine learning to classification problems, there's actually one other definition of J_{test} and J_{train} that is maybe even more commonly used.

Train/test procedure for classification problem

fraction of the test set and the fraction of the train set that the algorithm has misclassified.

$$\hat{y} = \begin{cases} 1 & \text{if } f_{\bar{w}, b}(\bar{x}^{(i)}) \geq 0.5 \\ 0 & \text{if } f_{\bar{w}, b}(\bar{x}^{(i)}) < 0.5 \end{cases}$$

count $\hat{y} \neq y$

$J_{\text{test}}(\bar{w}, b)$ is the fraction of the test set that has been misclassified.

$J_{\text{train}}(\bar{w}, b)$ is the fraction of the train set that has been misclassified.

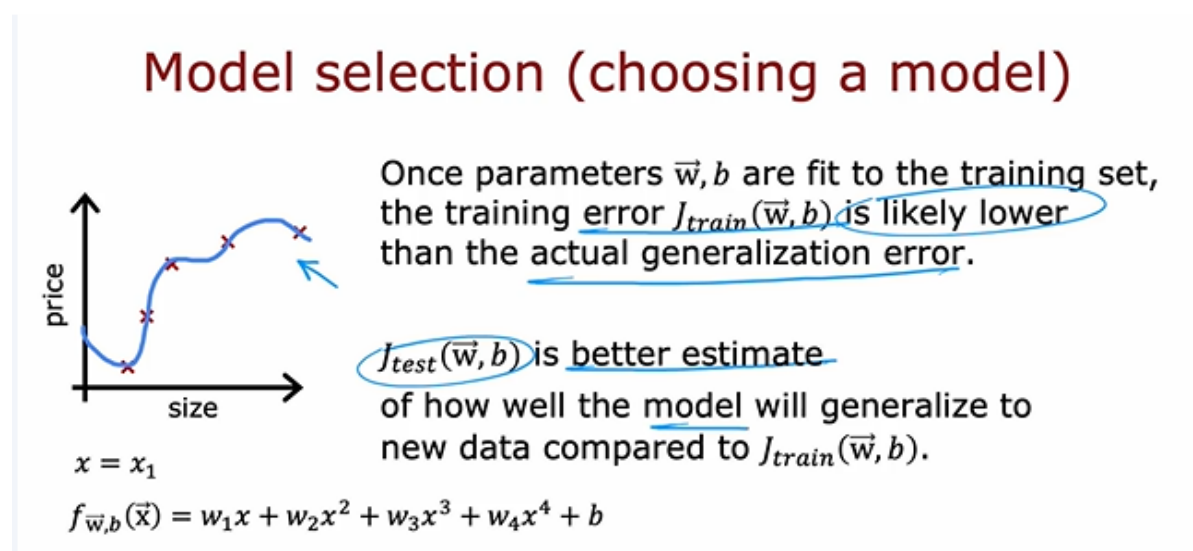
Which is instead of using the logistic loss to compute the test error and the training error to instead measure what the fraction of the test set, and the fraction of the training set that the algorithm has misclassified.

So specifically on the test set, you can have the algorithm make a prediction 1 or 0 on every test example. So, recall $\hat{y}$ we would predict us 1 if f of x is greater than equal 0.5, and zero if it's less than 0.5. And you can then count up in the test set the fraction of examples where $\hat{y}$ is not equal to the actual

ground truth label while in the test set. So concretely, if you are classifying handwritten digits 0, 1 binary classification loss, then J_{test} would be the fraction of that test set, where 0 was classified as 1 or 1, classified as 0. And similarly, J_{train} is a fraction of the training set that has been misclassified.

Taking a dataset and splitting it into a training set and a separate test set gives you a way to systematically evaluate how well your learning algorithm is doing. By computing both J_{test} and J_{train} , you can now measure how was doing on the test set and on the training set. This procedure is one step to what you'll be able to automatically choose what model to use for a given machine learning application. For example, if you're trying to predict housing prices, should you fit a straight line to your data, or fit a second order polynomial, or third order fourth order polynomial? It turns out that with one further refinement to the idea you saw in this video, you'll be able to have an algorithm help you to automatically make that type of decision well.

Model selection and training/cross validation/test sets



Model selection (choosing a model)

- $d=1$ 1. $f_{\bar{w},b}(\bar{x}) = w_1x + b \rightarrow w^{<1>}, b^{<1>} \rightarrow J_{test}(w^{<1>}, b^{<1>})$
 $d=2$ 2. $f_{\bar{w},b}(\bar{x}) = w_1x + w_2x^2 + b \rightarrow w^{<2>}, b^{<2>} \rightarrow J_{test}(w^{<2>}, b^{<2>})$
 $d=3$ 3. $f_{\bar{w},b}(\bar{x}) = w_1x + w_2x^2 + w_3x^3 + b \rightarrow w^{<3>}, b^{<3>} \rightarrow J_{test}(w^{<3>}, b^{<3>})$
 $\vdots$
 $d=10$ 10. $f_{\bar{w},b}(\bar{x}) = w_1x + w_2x^2 + \dots + w_{10}x^{10} + b \rightarrow J_{test}(w^{<10>}, b^{<10>})$
 Choose $w_1x + \dots + w_5x^5 + b$ $d=5$ $J_{test}(w^{<5>}, b^{<5>})$

How well does the model perform? Report test set error $J_{test}(w^{<5>}, b^{<5>})$?

The problem: $J_{test}(w^{<5>}, b^{<5>})$ is likely to be an optimistic estimate of generalization error (ie. $J_{test}(w^{<5>}, b^{<5>}) < \text{generalization error}$). Because an extra parameter d (degree of polynomial) was chosen using the test set.

w, b are overly optimistic estimate of generalization error on training data.

If a fitting a function to predict housing prices or some other regression problem, one model you might consider is to fit a linear model like this. This is a first-order polynomial and we're going to use $d = 1$ on this slide to denote fitting a one or first-order polynomial.

If you were to fit a model like this to your training set, you get some parameters, w and b , and you can then compute J tests to estimate how well does this generalize to new data? On this slide, I'm going to use $w^{<1>}, b^{<1>}$ to denote that these are the parameters you get if you were to fit a first order polynomial, a degree one, $d = 1$ polynomial.

Now, you might also consider fitting a second-order polynomial or quadratic model, so this is the model. If you were to fit this to your training set, you would get some parameters, $w^{<2>}, b^{<2>}$, and you can then similarly evaluate those parameters on your test set and get $J_{test}(w^{<2>}, b^{<2>})$, and this will give you a sense of how well the second-order polynomial does.

You can go on to try $d = 3$, that's a third order or a degree three polynomial that looks like this, and fit parameters and similarly get J_{test} . You might keep doing this until, say you try up to a 10th order polynomial and you end up with $J_{test}(w^{<10>}, b^{<10>})$. That gives you a sense of how well the 10th order polynomial is doing.

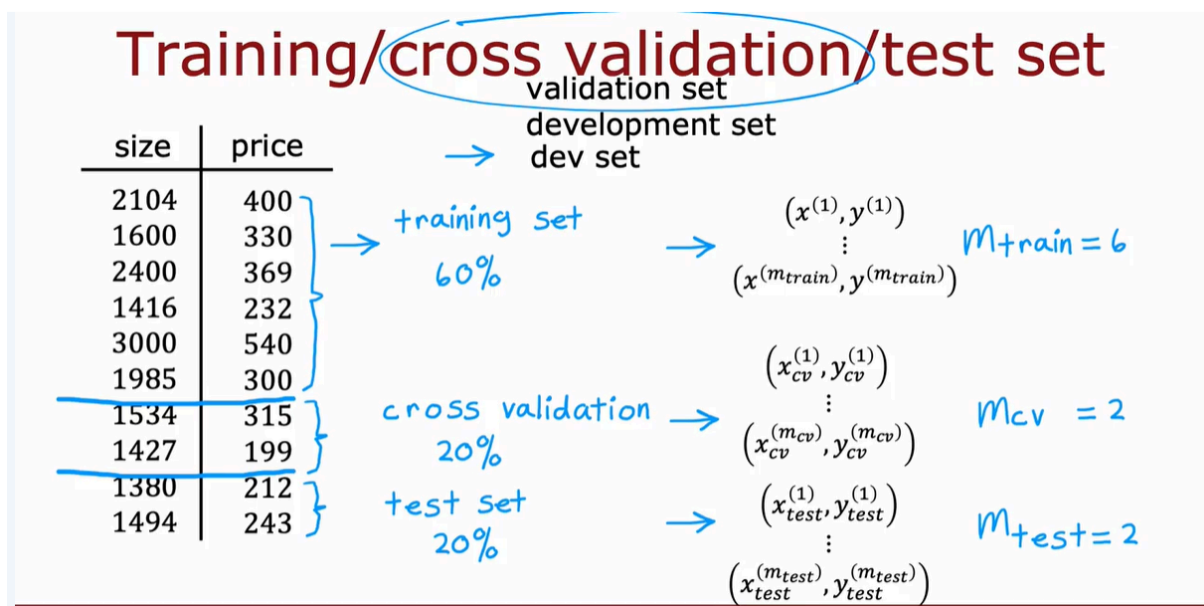
One procedure you could try, this turns out not to be the best procedure, but one thing you could try is, look at all of these J tests, and **see which one gives you the lowest value**. Say, you find that, $J_{test}(w^{<5>}, b^{<5>})$ turns out to be the lowest. If that's the case, then you might decide that the fifth order polynomial

$d = 5$ does best, and choose that model for your application. If you want to estimate how well this model performs, one thing you could do, but this turns out to be a slightly flawed procedure, is to report the test set error, $J_{test}(w^{<5>}, b^{<5>})$. The reason this procedure is flawed is $J_{test}(w^{<5>}, b^{<5>})$ is likely to be an optimistic estimate of the generalization error.

In other words, it is likely to be lower than the actual generalization error, and the reason is, in the procedure we talked about on this slide with basic fits, one extra parameter, which is d , the degree of polynomial, and we chose this parameter using the test set. On the previous slide, we saw that if you were to fit w, b to the training data, then the training data would be an overly optimistic estimate of generalization error. It turns out too, that if you want to choose the parameter d using the test set, then the test set J test is now an overly optimistic, that is lower than actual estimate of the generalization error.

The procedure on this particular slide is flawed and I don't recommend using this. Instead, if you want to automatically choose a model, such as decide what degree polynomial to use. Here's how you modify the training and testing procedure in order to carry out model selection. Whereby model selection, I mean choosing amongst different models, such as these 10 different models that you might contemplate using for your machine learning application.

Cross validation



The name cross-validation refers to that this is an extra dataset that we're going to use to check or cross check the validity or really the accuracy of different models.

I don't think it's a great name, but that is what people in machine learning have gotten to call this extra dataset. You may also hear people call this the validation set for short, it's just fewer syllables than cross-validation or in some applications, people also call this the development set. Means basically the same thing or for short. Sometimes you hear people call this the dev set, but all of these terms mean the same thing as cross-validation set. I personally use the term dev set the most often because it's the shortest, fastest way to say it but cross-validation is pretty used a little bit more often by machine learning practitioners.

Training/cross validation/test set

Training error:
$$J_{train}(\vec{w}, b) = \frac{1}{2m_{train}} \left[\sum_{i=1}^{m_{train}} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 \right]$$

Cross validation error:
$$J_{cv}(\vec{w}, b) = \frac{1}{2m_{cv}} \left[\sum_{i=1}^{m_{cv}} (f_{\vec{w}, b}(\vec{x}_{cv}^{(i)}) - y_{cv}^{(i)})^2 \right] \quad (\text{validation error, dev error})$$

Test error:
$$J_{test}(\vec{w}, b) = \frac{1}{2m_{test}} \left[\sum_{i=1}^{m_{test}} (f_{\vec{w}, b}(\vec{x}_{test}^{(i)}) - y_{test}^{(i)})^2 \right]$$

Onto these three subsets of the data training set, cross-validation set, and test set, you can then compute the training error, the cross-validation error, and the test error using these three formulas.

Whereas usual, none of these terms include the regularization term that is included in the training objective, and this new term in the middle, the cross-validation error is just the average over your m_{cv} cross-validation examples of the average say, squared error. This term (the yellow term), in addition to being called cross-validation error, is also commonly called the validation error for short, or even the development set error, or the dev error

Model selection

$$\begin{array}{ll}
 d=1 & 1. \quad f_{\vec{w},b}(\vec{x}) = w_1x + b \quad w^{<1>}, b^{<1>} \rightarrow J_{cv}(w^{<1>}, b^{<1>}) \\
 d=2 & 2. \quad f_{\vec{w},b}(\vec{x}) = w_1x + w_2x^2 + b \quad \rightarrow J_{cv}(w^{<2>}, b^{<2>}) \\
 d=3 & 3. \quad f_{\vec{w},b}(\vec{x}) = w_1x + w_2x^2 + w_3x^3 + b \quad \vdots \\
 \vdots & \vdots \\
 d=10 & 10. \quad f_{\vec{w},b}(\vec{x}) = w_1x + w_2x^2 + \dots + w_{10}x^{10} + b \quad J_{cv}(w^{<10>}, b^{<10>})
 \end{array}$$

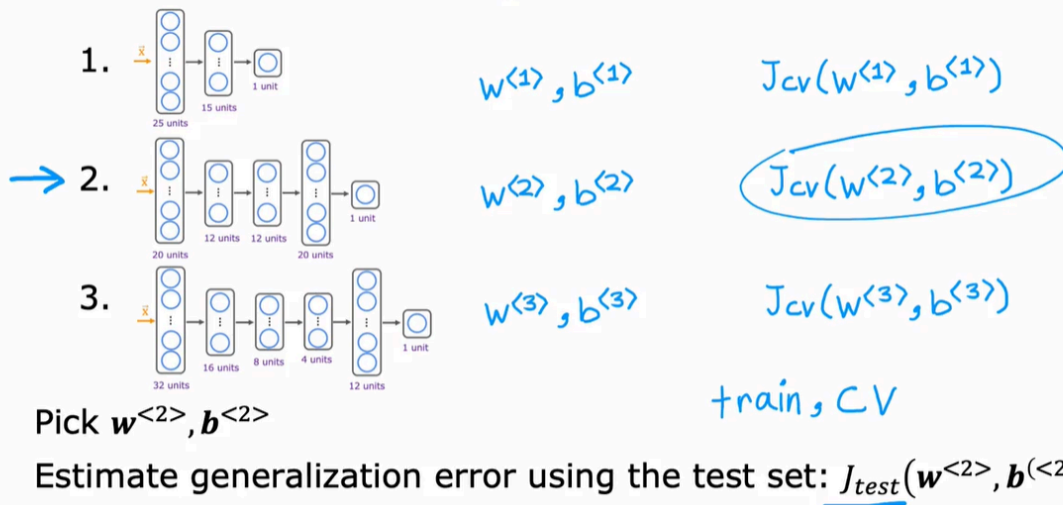
$\rightarrow$ Pick $w_1x + \dots + w_4x^4 + b$ ($J_{cv}(w^{<4>}, b^{<4>})$)
 Estimate generalization error using test the set: $J_{test}(w^{<4>}, b^{<4>})$

Armed with these three measures of learning algorithm performance, this is how you can then go about carrying out model selection. You can, with the 10 models, same as earlier on this slide, with $d = 1$, $d = 2$, all the way up to a 10th degree or the 10th order polynomial, you can then fit the parameters $w^{<1>}$, $b^{<1>}$. But instead of evaluating this on your test set, you will instead evaluate these parameters on your cross-validation sets and compute $J_{cv}(w^{<1>}, b^{<2>})$, and similarly, for the second model, we get $J_{cv}(w^{<2>}, b^{<2>})$, and all the way down to $J_{cv}(w^{<10>}, b^{<10>})$.

Then, in order to choose a model, you will look at which model has the lowest cross-validation error, and concretely, let's say that $J_{cv}(w^{<4>}, b^{<4>})$ as low as, then what that means is you pick this fourth-order polynomial as the model you will use for this application. Finally, if you want to report out an estimate of the generalization error of how well this model will do on new data. You will do so using that third subset of your data, the test set and you report out $J_{cv}(w^{<4>}, b^{<4>})$. You notice that throughout this entire procedure, you had fit these parameters using the training set.

You then chose the parameter d or chose the degree of polynomial using the cross-validation set and so up until this point, you've not fit any parameters, either w or b or d to the test set and that's why J_{test} in this example will be fair estimate of the generalization error of this model thus parameters $w^{<4>}, b^{<4>}$. This gives a better procedure for model selection and it lets you automatically make a decision like what order polynomial to choose for your linear regression model. This model selection procedure also works for choosing among other types of models.

Model selection – choosing a neural network architecture



Example:

There are three models, model 2 has the lowest cost value, → pick model 2 we have $w^{(2)}, b^{(2)}$

Then use $w^{(2)}, b^{(2)}$ to estimate generalization error